

CI/CD *Pipeline*

GitHub Actions, Vercel, Docker, DevOps

Dokumentacija CI/CD pipeline-a z **4 GitHub Actions workflow-i** (lint, typecheck, build, test), Vercel auto-deploy (3 environment-i), Docker multi-stage build, branch protection, secret management, monitoring integracija (Sentry), rollback procedure in **DevOps best practices**.

4

WORKFLOW-I

3

CI JOB-I

2

CRON JOBS

15

MIN PIPELINE

Kazalo vsebine

1. Uvod v CI/CD pipeline	3
1.1 Pipeline pregled	3
1.2 Workflow datoteke	3
2. CI pipeline - kvaliteta in testiranje	5
2.1 Quality job (lint + typecheck)	5
2.2 Build job (production build)	5
3. Vercel deployment	7
3.1 Deployment flow	7
3.2 Vercel konfiguracija	7
3.3 Cron jobs	7
3.4 Environment variables	8
4. Docker setup	9
4.1 Multi-stage Dockerfile	9
4.2 Docker commands	9
4.3 Docker security best practices	10
5. Branch protection in GitHub	11
5.1 Branch protection rules (main)	11
5.2 Konfiguracija v GitHub	11
6. Secret management	12
6.1 GitHub Secrets	12
6.2 Secret scanning	12

7. Monitoring in alerting integracija

13

7.1 Sentry release tracking

13

7.2 GitHub deployment status

13

7.3 CI/CD metrike

13

8. Rollback in recovery

15

8.1 Rollback postopek

15

8.2 Database rollback (Neon PITR)

15

9. DevOps best practices

16

9.1 Commit in branch strategija

16

9.2 Testing strategija

16

9.3 Deployment best practices

16

9.4 Security best practices

16

9.5 Performance best practices

16

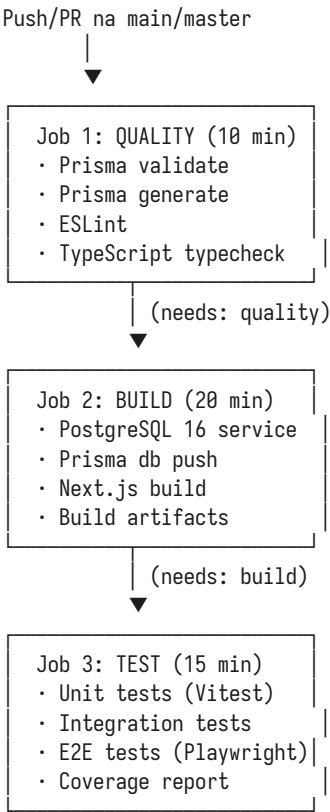
1. Uvod v CI/CD pipeline

RestaurantOS uporablja GitHub Actions za continuous integration in Vercel za continuous deployment. Pipeline je razdeljen v 4 workflow datoteke, ki avtomatsko preverjajo kvaliteto kode, buildajo aplikacijo, testirajo in deploy-ajo na produkcijo.

4	3	2	3
GitHub Actions workflow-i	CI job-i (quality, build, test)	cron jobs (Vercel)	deployment environment-i

1.1 Pipeline pregled

GitHub Actions Pipeline (ci.yml):



Vercel Deployment (avtomatsko):
Push na main → Vercel build → Preview URL → Promote to Production
Push na PR → Vercel preview deployment

1.2 Workflow datoteke

Datoteka	Trigger	Namen	Trajanje
.github/workflows/ci.yml	Push/PR na main	Lint, typecheck, build, test	~15 min
.github/workflows/db-push.yml	Manual/Schedule	Prisma db push na staging	~3 min

Datoteka	Trigger	Namen	Trajanje
.github/workflows/test-app.yml	Push na PR	App-level integration test-i	~10 min
.github/workflows/test-live.yml	Schedule (hourly)	Live production health check	~2 min

Tabela 1.1: GitHub Actions workflow datoteke

2. CI pipeline - kvaliteta in testiranje

2.1 Quality job (lint + typecheck)

Prvi job v pipeline-u preverja osnovno kakovost kode. Zažene se ob vsakem push-u ali PR-ju na main/master.

```
# .github/workflows/ci.yml (quality job)

quality:
  name: Lint & Typecheck
  runs-on: ubuntu-latest
  timeout-minutes: 10
  env:
    DATABASE_URL: "postgresql://ci:ci@localhost:5432/ci?schema=public"
  steps:
    - uses: actions/checkout@v4

    - name: Setup Bun
      uses: oven-sh/setup-bun@v2
      with:
        bun-version: latest

    - name: Setup Node.js
      uses: actions/setup-node@v4
      with:
        node-version: 22

    - name: Install dependencies
      run: bun install --frozen-lockfile

    - name: Validate Prisma schema
      run: bunx prisma validate --schema prisma/schema.prisma

    - name: Generate Prisma client
      run: bunx prisma generate

    - name: ESLint
      run: bun run lint

    - name: TypeScript typecheck
      run: bunx tsc --noEmit
```

2.2 Build job (production build)

Drugi job build-a aplikacijo z realno PostgreSQL bazo (Docker service). Preverja, da build uspešno gre skozi.

```
# .github/workflows/ci.yml (build job)

build:
  name: Build
  runs-on: ubuntu-latest
  timeout-minutes: 20
  needs: quality
  env:
    DATABASE_URL: "postgresql://ci:ci@localhost:5432/ci?schema=public"
    NEXTAUTH_SECRET: "ci-test-secret-do-not-use-in-prod"
    FURS_ENV: "test"
```

```
FURS_ALLOW_SIMULATION: "true"
services:
  postgres:
    image: postgres:16-alpine
    env:
      POSTGRES_USER: ci
      POSTGRES_PASSWORD: ci
      POSTGRES_DB: ci
    ports:
      - 5432:5432
    options: >-
      --health-cmd pg_isready
      --health-interval 10s
      --health-timeout 5s
      --health-retries 5
  steps:
    - uses: actions/checkout@v4
    - name: Setup Bun
      uses: oven-sh/setup-bun@v2
    - name: Install dependencies
      run: bun install --frozen-lockfile
    - name: Generate Prisma client
      run: bunx prisma generate
    - name: Push database schema
      run: bunx prisma db push
    - name: Build Next.js
      run: bun run build
    - name: Upload build artifacts
      uses: actions/upload-artifact@v4
      with:
        name: build-output
        path: .next/
        retention-days: 7
```

3. Vercel deployment

Vercel avtomatsko deploy-a aplikacijo ob vsakem push-u na GitHub. Obstajajo 3 deployment environment-i: produkcija, preview (za PR-je) in development.

3.1 Deployment flow

Automatic deployment flow:

1. Developer push-a na feature branch
 - Vercel ustvari PREVIEW deployment
 - URL: `https://restaurantos-<branch>.vercel.app`
 - Uporablja STAGING environment variables
2. Developer odpre Pull Request
 - Vercel ustvari preview z PR specifičnim URL
 - Avtomatski komentar na PR z preview linkom
 - Reviewer lahko testira pred merge
3. PR je merged v main
 - Vercel build-a in deploy-a na PRODUCTION
 - URL: `https://restaurantos.app` (po DNS konfiguraciji)
 - Uporablja PRODUCTION environment variables
 - Sentry release tag se nastavi
4. Rollback (če nekaj gre narobe)
 - Vercel Dashboard → Deployments
 - Izberi prejšnji uspešni deployment
 - Klikni "Promote to Production"
 - Stara verzija postane aktivna v <1 min

3.2 Vercel konfiguracija

vercel.json

```
{
  "crons": [
    {
      "path": "/api/scheduled-emails/process",
      "schedule": "0 2 * * *" // vsak dan ob 2:00
    },
    {
      "path": "/api/cron/outbox",
      "schedule": "0 3 * * *" // vsak dan ob 3:00
    }
  ],
  "headers": [
    {
      "source": "/(.*)",
      "headers": [
        { "key": "X-Content-Type-Options", "value": "nosniff" },
        { "key": "X-Frame-Options", "value": "SAMEORIGIN" },
        { "key": "Referrer-Policy", "value": "strict-origin-when-cross-origin" }
      ]
    }
  ]
}
```

3.3 Cron jobs

Cron	Path	Namig	Namen
0 2 * * *	/api/scheduled-emails/process	Dnevno ob 2:00	Pošiljanje scheduled emailov (poročila, opomniki)
0 3 * * *	/api/cron/outbox	Dnevno ob 3:00	Procesiranje outbox queue (async eventi)

Tabela 3.1: Vercel cron jobs

3.4 Environment variables

Spremenljivka	Produkcija	Preview	Development
DATABASE_URL	Neon prod URL	Neon staging URL	Neon dev ali PGLite
NEXTAUTH_SECRET	strong-random	strong-random	random
FURS_ENV	production	test	test
FURS_CERT_PATH	./certs/furs-prod.p12	./certs/furs-test.p12	-
STRIPE_SECRET_KEY	sk_live_...	sk_test_...	sk_test_...
SENTRY_DSN	prod DSN	staging DSN	-
NEXT_PUBLIC_VAPID_PUBLIC_KEY	live key	test key	-

Tabela 3.2: Environment variables po environment-u

4. Docker setup

RestaurantOS ima Dockerfile za self-hosted deploy (alternative Vercel-u). Uporablja multi-stage build za manjši image in boljšo varnost.

4.1 Multi-stage Dockerfile

```
# Dockerfile (multi-stage, non-root)

# Stage 1: Base
FROM node:20-alpine AS base
RUN apk add --no-cache libc6-compat

# Stage 2: Dependencies
FROM base AS deps
WORKDIR /app
COPY package.json package-lock.json ./
RUN npm ci --only=production

# Stage 3: Build
FROM base AS builder
WORKDIR /app
COPY --from=deps /app/node_modules ./node_modules
COPY . .
RUN npx prisma generate
RUN npm run build

# Stage 4: Runner (production)
FROM base AS runner
WORKDIR /app
ENV NODE_ENV=production

# Non-root user (security)
RUN addgroup --system --gid 1001 nodejs
RUN adduser --system --uid 1001 nextjs

# Copy build output
COPY --from=builder /app/public ./public
COPY --from=builder --chown=nextjs:nodejs /app/.next/standalone ./
COPY --from=builder --chown=nextjs:nodejs /app/.next/static ./next/static

# Switch to non-root
USER nextjs

EXPOSE 3000
ENV PORT=3000
ENV HOSTNAME="0.0.0.0"

CMD ["node", "server.js"]
```

4.2 Docker commands

```
# Build image:
docker build -t restaurantos .

# Run container:
docker run -p 3000:3000 \
  -e DATABASE_URL="postgresql://..." \
  -e NEXTAUTH_SECRET="..." \
  -e FURS_ENV="test" \
```

restaurantos

Run with docker-compose:

```
docker-compose up -d
```

Tag and push to registry:

```
docker tag restaurantos ghcr.io/markec12345678/restaurantos:latest
```

```
docker push ghcr.io/markec12345678/restaurantos:latest
```

Pull and run on server:

```
docker pull ghcr.io/markec12345678/restaurantos:latest
```

```
docker run -d --name restaurantos -p 3000:3000 \
```

```
--env-file .env.production \
```

```
ghcr.io/markec12345678/restaurantos:latest
```

4.3 Docker security best practices

- **Non-root user** - aplikacija teče kot `nextjs` (UID 1001), ne root
- **Multi-stage build** - build artifacts ločeni od production image-a
- **Read-only filesystem** - `--read-only` flag v production
- **No secrets in image** - vse skozi environment variables
- **Image scanning** - Trivy v CI pipeline za ranljivosti
- **Minimal base image** - `node:20-alpine` (manj attack surface)

5. Branch protection in GitHub

GitHub branch protection rules zagotavljajo, da koda na main veji vedno prestane vse preglede in teste.

5.1 Branch protection rules (main)

Pravilo	Nastavitev	Namen
Require pull request	✔ Enabled	Niž direktnih push-ov na main
Required reviewers	Min 1	Code review pred merge
Dismiss stale reviews	✔ Enabled	Nov commit invalidira stare reviews
Require status checks	✔ Enabled	CI mora pasti pred merge
Required CI checks	quality + build	Lint, typecheck, build
Require branches up-to-date	✔ Enabled	Merge z latest main pred merge
Require signed commits	✘ Disabled	Zaenkrat ne (lahko omogoči kasneje)
Require linear history	✔ Enabled	Squash merge (čist history)
Allow force pushes	✘ Disabled	Nikoli na main
Allow deletions	✘ Disabled	Nikoli ne briši main

Tabela 5.1: Branch protection rules za main

5.2 Konfiguracija v GitHub

```
# GitHub Settings → Branches → Branch protection rules
# (ročno nastavi v GitHub UI)

Repository: markec12345678/restaurantos
Settings → Branches → Add rule

Branch name pattern: main
☐ Require pull request reviews before merging
  Minimum required reviewers: 1
    ☐ Dismiss stale pull request approvals when new commits are pushed
    ☐ Require review from Code Owners
☐ Require status checks to pass before merging
  ☐ Require branches to be up to date before merging
  Status checks: quality, build
☐ Require conversation resolution before merging
☐ Require linear history
☐ Do not allow bypassing the above settings
```

6. Secret management

Tajni ključi se hranijo na 3 mestih: GitHub Secrets (CI/CD), Vercel Environment Variables (runtime) in .env (lokalni razvoj). Nikoli ne v kodi ali Git-u.

6.1 GitHub Secrets

Secret	Namig	Uporabljen v
NEON_DATABASE_URL	Neon production connection string	db-push.yml workflow
NEON_STAGING_DATABASE_URL	Neon staging connection string	db-push.yml workflow
VERCEL_TOKEN	Vercel deploy token	test-live.yml workflow
SENTRY_AUTH_TOKEN	Sentry release token	ci.yml (release creation)
STRIPE_SECRET_KEY	Stripe live secret	Samo v Vercel (ne v CI)
FURS_CERT_PASSWORD	FURS cert geslo	Samo v Vercel (ne v CI)

Tabela 6.1: GitHub Secrets

6.2 Secret scanning

- **gitleaks** - skenira kodo za tajne ključe ob vsakem commit-u
- **GitHub Secret Scanning** - vgrajeno v GitHub (settings → Security → Secret scanning)
- **Dependabot** - preverja odvisnosti za znane ranljivosti

```
# gitleaks v CI (pre-commit hook):
# .pre-commit-config.yaml
repos:
  - repo: https://github.com/gitleaks/gitleaks
    rev: v8.18.0
    hooks:
      - id: gitleaks
```

```
# GitHub Dependabot:
# .github/dependabot.yml
version: 2
updates:
  - package-ecosystem: "npm"
    directory: "/"
    schedule:
      interval: "weekly"
    open-pull-requests-limit: 10

  - package-ecosystem: "github-actions"
    directory: "/"
    schedule:
      interval: "weekly"
```

7. Monitoring in alerting integracija

CI/CD pipeline je integriran z Sentry za error tracking in GitHub Status za deployment status.

7.1 Sentry release tracking

```
# Sentry release creation v CI (po uspešnem build-u):
# .github/workflows/ci.yml (dodaj v build job)

- name: Create Sentry release
  if: github.ref == 'refs/heads/main'
  uses: getsentry/action-release@v1
  env:
    SENTRY_AUTH_TOKEN: ${ secrets.SENTRY_AUTH_TOKEN }
  with:
    environment: production
    version: ${ github.sha }
    set_commits: skip
    projects: restaurantos

# Sentry source maps upload (za stack traces):
- name: Upload Sentry source maps
  if: github.ref == 'refs/heads/main'
  run: |
    npx sentry-cli sourcemaps upload --release=${ github.sha } .next/static/chunks/
```

7.2 GitHub deployment status

```
# GitHub Deployment API (avtomatsko preko Vercel):
# Ko Vercel deploy-a, ustvari GitHub Deployment:

POST /repos/{owner}/{repo}/deployments
{
  "ref": "main",
  "environment": "production",
  "description": "Vercel deployment",
  "required_contexts": ["ci/quality", "ci/build"]
}

# Status se posodobi:
# → "pending" (build in progress)
# → "success" (deploy successful)
# → "failure" (deploy failed)
# → "error" (build error)

# Vidno v GitHub UI: Repository → Deployments
```

7.3 CI/CD metrike

Metrika	Cilj	Merenje	Alert če
Build duration	< 20 min	GitHub Actions	> 30 min
Test pass rate	100%	CI test job	< 95%
Deploy frequency	Dnevno	Vercel + GitHub	Ni deploy-a 3 dni
Lead time (commit → prod)	< 1 ura	GitHub timestamps	> 4 ure

Metrika	Cilj	Merenje	Alert će
MTTR (mean time to recover)	< 1 ura	Incident log	> 4 ure
Change failure rate	< 10%	Failed deploys / total	> 20%

Tabela 7.1: CI/CD metrike in alerti

8. Rollback in recovery

Rollback je hiter in enostaven preko Vercel dashboard-a. Vsak deployment ima svoj URL in je takoj promoviran v produkcijo.

8.1 Rollback postopek

Rollback preko Vercel Dashboard:

1. <https://vercel.com/robertpezdir12/restaurantos/deployments>
2. Poišči zadnji "Ready" deployment PRED incidentom
3. Klikni "..." → "Promote to Production"
4. Potrdi - nov production deployment = stara verzija
5. Preveri /api/health
6. Preveri Sentry (ali so errorji izginili)
7. Komuniciraj strankam (status page update)

Rollback preko Vercel CLI:

```
vercel rollback <deployment-url>
```

Po rollback:

- Preveri /api/health → 200 OK
- Preveri Sentry → novi errorji?
- Preveri DB → pravilni podatki?
- Komuniciraj na Slack #incidents
- Napiši post-mortem v 24h

8.2 Database rollback (Neon PITR)

Neon Point-in-Time Recovery:

1. Neon Console → ep-solitary-term-anhf13uv
2. Restore → Point in Time
3. Izberi časovno točko (pred incidentom)
4. Nov branch se ustvari z restored stanjem
5. Testiraj na branch-u
6. Posodobi DATABASE_URL v Vercel (promote branch to main)

PRIPOROČILO: Pred restore naredi snapshot trenutnega stanja!

9. DevOps best practices

9.1 Commit in branch strategija

- **Trunk-based development** - kratke feature branch-e (1-2 dni), pogosto merge v main
- **Squash merge** - čist history, en commit per PR
- **Conventional commits** - feat:, fix:, chore:, docs:, test:, refactor:
- **PR description** - kaj, zakaj, kako testirati, screenshot-i
- **Small PRs** - <400 vrstic za lažji review

9.2 Testing strategija

- **Test pyramid** - 70% unit, 20% integration, 10% E2E
- **CI run-a vse teste** - pred vsakim merge v main
- **Parallel test execution** - Playwright test-i v paralelnih runnerjih
- **Test data** - seedani demo podatki v testnem okolju
- **Flaky test management** - quarantine + fix v 24h

9.3 Deployment best practices

- **Deploy v off-peak** - 22:00-06:00 (manj strank aktivnih)
- **Blue-green deploy** - Vercel to omogoča (instant rollback)
- **Database migrations** - backward compatible (dodaj stolpec, ne briši)
- **Feature flags** - za postopno rollout (LaunchDarkly ali custom)
- **Canary releases** - 10% traffic → 50% → 100%

9.4 Security best practices

- **Least privilege** - CI/CD ima samo potrebne pravice
- **Rotate secrets** - vsako leto (NEXTAUTH_SECRET, STRIPE keys)
- **Audit access** - GitHub audit log + Vercel access log
- **Dependency scanning** - Dependabot + npm audit v CI
- **Container scanning** - Trivy za Docker images

9.5 Performance best practices

- **Build caching** - GitHub Actions cache + Vercel build cache
- **Bundle analysis** - @next/bundle-analyzer v CI
- **Lighthouse CI** - performance regression detection
- **Bundle size limit** - <2MB gzipped (warning), <3MB (error)

CI/CD ZAKLJUČEK

RestaurantOS ima robusten CI/CD pipeline z GitHub Actions (lint, typecheck, build, test) in Vercel (auto-deploy, preview, rollback). Pipeline zagotavlja kakovost kode (0 errorjev pred merge), hitro feedback zanko (~15 min) in zanesljiv deploy z instant rollback. Priporočena DORA metrika: deploy frequency dnevno, lead time <1h, MTTR <1h, change failure rate <10%.